

# 答疑汇总与学习建议 #3

## 1.答疑汇总

程序在读入过程中出现了问题

除非你需要把空格作为匹配格式的一部分，请不要在scanf的格式串中使用空格或者回车。例如

```
1. scanf("%d ", &number);
2. scanf("%d\n", &number);
```

是不可取的。因为输入数据不一定每个数字后面都有空格，或者\n。

scanf会自动过滤回车、空格、制表符号，无需你在格式串里面加上它们。如果你上一行本应该输入10个数字，但是你只输入了9个，scanf会读取下一行的第一个数字。

整除和除

两个整型相除是整除。例如：

```
int a = 3, b = 2;
a / b;
```

第二个表达式的结果是整型的1. 如果你需要的不是整除，请将a或者b任意一个类型在表达式中强制转化为浮点类型。如：

```
int a = 3, b = 2;
a / (double)b;
```

该表达式的结果是浮点数的1.5。

表达式的类型与参与运算的量有关。具体参考C语言资料。

栈内存数组、变量没有初始化

定义在函数内部的数组、变量在C语言中不会初始化。因此，他们的值是未知的（往往是此处物理地址在内存中原来储存的数据），因此你需要通过初始化（往往是0）来重置它。

数组、变量未初始化是一个常见的错误。

提示：在微软Visual C++ Debug中，未初始化的栈内存每个字节会被统一设置为0xCD。

栈内存有限

有些同学在栈内存中创建了过大的数组，引起了编译错误或者运行时错误。

```
int main() {
    int bigArray[0x7fffffff], i;
    for (i = 0; i < 0x7fffffff; ++i) {
        bigArray[i] = i;
    }
    return 0;
}
```

由于栈内存有限，请把大数组移到函数定义之外。此时大数组占用的是静态内存空间。

## 2.学习建议

### 使用断言（assert）调试

```
#include <assert.h>
#include <stdlib.h>

void fill_array(int* begin, int n, int val) {
    int i;
    assert(n > 0 && begin);
    for (i = 0; i < n; ++i) {
        *(begin + i) = val;
    }
}

int main() {
    fill_array(NULL, 10, 0);
    return 0;
}
```

断言使用方法像一个函数，但它实际上是一个宏。括号里面放断言的表达式。断言穿插在程序中，假定你所提供的表达式必定为真。假如表达式为假，那么断言失败，程序运行不符合预期，程序将会强制停止。

### 遇到问题先自己动手

碰到“答案错误”的问题，可能有的同学一下子就摸不到头脑了，束手无策，转向寻求他人的帮助。在遇到问题时，建议同学们通过调试工具自行查找程序运行不符合预期的地方。可以通过制造除测试样例之外的测试数据测试你的程序，对比程序运行的结果和你预想的结果。

例如求一个数字出现次数最多的质因子这个题。题目给了两个测试数据，许多同学的程序都得出了正确的结果。但是提交到OJ上却返回答案错误。这时候你可以自己制造一些测试数据。例如147，84563等，用计算器查看它们的质因子，与你的程序的结果进行对比。如果发现不同，恭喜你，根据这个样例调试你的程序，你将会找到程序错在哪里。

调试是一项艰苦的工作，要有这个毅力去做，不要依靠别人的帮助。调试的过程可能会很漫长，可能数个小时过后都不能找到问题所在。但是只有这样才能培养起调试的能力、发现程序Bug的能力。

### 开始用i之前记得赋初值

一个常见的错误是用过的i在下次使用前没有重置为0：

```
int i = 0, sum = 0;
for (; i < n; ++i) {
    scanf("%d", &array[i]);
}
for (; i < n; ++i) {
    sum += array[i];
}
```

再次使用i前务必记得重置i的值。有两种办法可以避免犯这种错误。一是在for语句头部重置i：

```
for (i = 0; i < n; ++i)
```

另一种办法是即用即时定义。但是c语言只能在函数开始时定义变量，这种写法只支持c++。

```
for (int i = 0; i < n; ++i)
```

## 3.拓展知识

---

### OJ如何进行判题

当你的代码文本上传到OJ上之后，OJ会使用它自己的编译器编译你的代码。这是整个过程的第一步。如果你的代码无法被编译（通常是因为你的代码之中存在语法错误），OJ会向你抱怨“编译失败”。

这之后OJ会测试你的程序的正确性。OJ对每个题目都准备了许多组测试数据，一组测试数据包括输入与期望的正确输出。测试数据一般不会包含样例数据，样例数据只是帮助你理解题目、初步测试程序正确性而使用的。所以说，你的程序通过了样例数据并不意味着在OJ上一定能得分。OJ挨个把每组数据的输入数据提供给你的程序，等待你的程序的输出数据。如果在一定的时间内没有等到你的程序执行完成，OJ会报告运行时间超时。如果你的程序崩溃了，会报告运行时错误。如果你的程序用了太多的内存，会报告内存超出限制。万幸，你的程序安全地完成执行，进入到下一步。

下一步OJ会对比你的程序的输出和测试数据期望的正确的输出。如果对比发生不同，OJ汇报“答案错误”。否则，OJ汇报“答案正确”，这道题就完成了。